CESI
ÉCOLE D'INGÉNIEURS

Stockage & Big Data

CER | Requêtage

Mercredi 28 octobre 2026

Mazou Marine
Fisa Info 25–28
CESI

TABLE DES MATIÈRES

Étude de cas : Bee patient **2**
AAV . . . 2
Mots clés . . . 2
Contexte . . . 2
Problématique . . . 3
Contraintes . . . 3
Généralisation . . . 3
Livrables . . . 3
Pistes de solutions . . . 4
Plan d'actions . . . 4

1 Définitions des mots clés **5**

2 Les familles de bases NoSQL **7**
2.1 Clé-valeur . . . 7
2.2 Documents . . . 7
2.3 Colonnes . . . 8
2.4 Graphes . . . 8
2.5 Tableau comparatif . . . 9

3 SGBDR et NoSQL **10**
3.1 ACID et BASE . . . 10
3.2 Le théorème CAP . . . 10
3.3 Comparaison . . . 12
3.4 Choisir le stockage adapté à un besoin métier . . . 12

4 Hive et MapReduce **14**
4.1 Architecture de Hive . . . 14
4.2 Fonctionnement de MapReduce . . . 15
4.3 Exemple : nombre de décès par région en 2019 . . . 15

5 Optimiser le modèle physique dans Hive **17**
5.1 Partitionnement . . . 17
5.2 Bucketing . . . 18
5.3 Formats colonnes et compression . . . 18
5.4 Statistiques, vectorisation et optimiseur . . . 19
5.5 Vues matérialisées . . . 19
5.6 Synthèse des leviers . . . 20

6 Application au CHU : modèle physique dans Hive **21**
6.1 Volumes et choix par table . . . 21
6.2 Création des tables . . . 23
6.3 Chargement depuis les tables de transit . . . 24
6.4 Requêtes des besoins . . . 24

6.5 Vue matérialisée 26
6.6 Protocole de mesure 27

7 Intégrer des données dans une base NoSQL avec un ETL 28
7.1 Concevoir la clé de ligne 28
7.2 Le job Talend 28

Conclusion 30

Capitalisation 31

Ressources 32

ÉTUDE DE CAS : BEE PATIENT

AAV

2 Distinguer les différents types de bases NoSQL (clé-valeur, orientée documents, orientée colonnes, orientée graphes) dans leur organisation et dans leurs cas d'usage

2 Comparer les propriétés des SGBDR et des bases NoSQL (propriétés ACID vs BASE, théorème CAP)

3 Utiliser un ETL pour intégrer des données dans une base NoSQL

4 Sélectionner le type de stockage adapté à un besoin métier

3 Utiliser MapReduce pour extraire des informations

4 Optimiser le modèle physique (partitionnement, bucket)

MOTS CLÉS

- NoSQL (clé-valeur, documents, colonnes, graphes)
- ACID / BASE
- Théorème CAP
- Hive
- MapReduce
- Modèle physique
- Partitionnement
- Bucketing
- Vue matérialisée

CONTEXTE

Une réunion avec un panel d'utilisateurs (médecins, personnels infirmiers, chercheurs et administrateurs d'établissements) a revalidé toutes les demandes d'analyse. Le modèle conceptuel ne change pas et l'ETL n'évolue que pour les calculs d'agrégation. L'équipe lance donc l'implémentation du modèle physique de l'entrepôt dans Hive, le remplissage des tables et le requêtage des analyses. Les données sources seront mises à jour tous les trois mois, au même rythme que la définition de nouvelles analyses.

PROBLÉMATIQUE

Comment implémenter et optimiser le modèle physique de l'entrepôt dans Hive pour que les analyses demandées, portant sur des millions de lignes, s'exécutent en moins de deux minutes ?

CONTRAINTES

- Temps de réponse maximal de 2 minutes par analyse
- Volumes importants : plusieurs millions de lignes et requêtes à plus de 3 jointures
- Délai de 3 jours pour produire les analyses
- Mise à jour des sources et des analyses tous les 3 mois
- Modèle conceptuel inchangé
- Stockage sur Hive et l'écosystème Hadoop

GÉNÉRALISATION

- Optimisation du modèle physique
- Choix du type de stockage
- Performance des requêtes

LIVRABLES

- Modèle physique de l'entrepôt implémenté dans Hive (tables, partitions, buckets)
- Requêtes des analyses demandées et mesure de leurs temps de réponse
- Comparaison des types de stockage (SGBDR et familles NoSQL)

PISTES DE SOLUTIONS

- Partitionner les tables de faits sur une colonne souvent filtrée, comme l'année.
- Répartir les tables en buckets sur les clés de jointure.
- Précalculer les agrégations dans des vues matérialisées.
- Utiliser un format de stockage en colonnes (ORC, Parquet).
- Comparer SGBDR et bases NoSQL pour choisir le stockage adapté à chaque besoin.

PLAN D'ACTIONS

- Définir les mots clés, étudier les ressources et réaliser le workshop.
- Présenter les familles de bases NoSQL et leurs cas d'usage.
- Comparer SGBDR et NoSQL : propriétés ACID et BASE, théorème CAP.
- Étudier le fonctionnement de Hive et de MapReduce.
- Étudier les leviers d'optimisation : partitionnement, bucketing, vues matérialisées, formats de stockage.
- Implémenter le modèle physique de l'entrepôt du CHU dans Hive et charger les tables.
- Écrire les requêtes des analyses et mesurer leurs temps de réponse avant et après optimisation.

1 DÉFINITIONS DES MOTS CLÉS

NoSQL: *Not only SQL*. Famille de bases de données qui n'utilisent pas le modèle relationnel en tables liées. Elles abandonnent souvent le schéma fixe et une partie des garanties transactionnelles pour passer à l'échelle sur plusieurs serveurs. On distingue quatre familles :

- **clé-valeur** : une clé unique donne accès à une valeur opaque (Redis) ;
- **documents** : la valeur est un document structuré (JSON, BSON) dont on peut interroger les champs (MongoDB) ;
- **colonnes** : les données sont rangées par familles de colonnes, ligne par ligne, avec des colonnes variables d'une ligne à l'autre (HBase, Cassandra) ;
- **graphes** : les données sont des nœuds reliés par des relations, elles-mêmes porteuses de propriétés (Neo4j).

ACID / **BASE**: Deux ensembles de propriétés qui décrivent le comportement d'une base face aux écritures.

- **ACID** (SGBD relationnels) : **A**tomicité (une transaction est faite entièrement ou pas du tout), **C**ohérence (les contraintes restent respectées), **I**solation (les transactions simultanées ne se gênent pas), **D**urabilité (une transaction validée n'est jamais perdue).
- **BASE** (beaucoup de bases NoSQL) : **B**asically **A**vailable (la base répond toujours), **S**oft state (l'état peut changer sans nouvelle écriture, le temps que les copies se synchronisent), **E**ventual consistency (toutes les copies finissent par être identiques).

Théorème CAP: Énoncé par Eric Brewer en 2000. Un système distribué ne peut pas garantir en même temps les trois propriétés suivantes :

- **C**ohérence (*Consistency*) : tous les nœuds voient la même donnée au même moment ;
- **D**isponibilité (*Availability*) : toute requête reçoit une réponse ;
- **T**olérance au partitionnement (*Partition tolerance*) : le système continue de fonctionner si le réseau coupe le cluster en morceaux.

Sur un cluster, une coupure réseau finit toujours par arriver : le vrai choix se fait entre cohérence et disponibilité pendant la coupure (partie 3.2).

Hive: Entrepôt de données de l'écosystème Hadoop, créé chez Facebook puis confié à la fondation Apache. Il permet d'interroger en **HiveQL**, un langage proche de SQL, des fichiers stockés dans HDFS. Le **metastore** garde la description des tables (colonnes, format, emplacement, partitions) ; chaque requête est traduite en tâches distribuées (MapReduce, Tez ou Spark). Hive est conçu pour l'analyse de gros volumes en lecture, pas pour des transactions rapides.

MapReduce: Modèle de programmation et moteur de calcul distribué de Hadoop, issu d'un article de Google (2004). Un traitement s'écrit en deux fonctions :

- **map** : lit chaque enregistrement et produit des paires (clé, valeur) ;
- **reduce** : reçoit toutes les valeurs d'une même clé et les agrège.

Entre les deux, le framework trie et regroupe les paires par clé (*shuffle and sort*). Les calculs sont lancés sur les nœuds qui portent déjà les données.

Modèle physique: Traduction du modèle dimensionnel dans un moteur précis. Il fixe, pour chaque fait et chaque dimension, les types des colonnes, le format de stockage (texte, ORC, Parquet), la compression, le découpage en partitions et en buckets, et les structures annexes (vues matérialisées, statistiques). Le modèle dimensionnel dit **quoi** stocker ; le modèle physique dit **comment** le stocker pour que les requêtes soient rapides.

Partitionnement: Découpage d'une table selon la valeur d'une ou plusieurs colonnes. Dans Hive, chaque valeur devient un **répertoire** HDFS (`annee=2019/`). Une requête qui filtre sur cette colonne ne lit que les répertoires utiles : c'est l'élagage de partitions (*partition pruning*).

Bucketing: Répartition des lignes d'une table (ou de chaque partition) dans un nombre fixe de **fichiers**, les buckets, selon un hachage d'une colonne : `bucket = hash(colonne) mod n`. Deux tables découpées de la même façon sur leur clé de jointure peuvent être jointes bucket par bucket, sans redistribuer toutes les données.

Vue matérialisée: Vue dont le résultat est calculé et **stocké** sur disque, au lieu d'être recalculé à chaque appel. Elle sert à précalculer des agrégations coûteuses. Dans Hive (version 3 et plus), l'optimiseur peut réécrire automatiquement une requête pour lire la vue à la place des tables d'origine. La vue doit être reconstruite quand les données sources changent.

2 LES FAMILLES DE BASES NOSQL

Les bases NoSQL sont nées des besoins du web (Google, Amazon, Facebook) : stocker des volumes qu'un seul serveur ne tient plus, et répondre à des millions d'utilisateurs. Chaque famille organise les données autrement, et chacune est efficace pour un type d'accès précis. On modélise donc une base NoSQL **à partir des requêtes** à servir, et non à partir des données.

2.1 CLÉ-VALEUR

La base est un grand dictionnaire : une clé unique donne une valeur que la base ne sait pas lire (texte, nombre, objet sérialisé). Les seules opérations sont lire, écrire et supprimer par clé.

- **Produits** : Redis (en mémoire), Amazon DynamoDB, Riak.
- **Points forts** : temps de réponse de l'ordre de la milliseconde, répartition simple sur plusieurs serveurs.
- **Limites** : aucune requête sur le contenu de la valeur, pas de jointure.
- **Cas d'usage** : cache, sessions d'utilisateurs, compteurs, files d'attente.

2.2 DOCUMENTS

La valeur est un document JSON ou BSON, avec des champs imbriqués et des tableaux. La base connaît la structure du document : on peut filtrer, indexer et agréger sur ses champs. Deux documents d'une même collection peuvent avoir des champs différents.

- **Produits** : MongoDB, CouchDB, Elasticsearch (orienté recherche).
- **Points forts** : schéma souple, un objet métier complet lu en une seule fois.
- **Limites** : jointures limitées, données dupliquées entre documents.
- **Cas d'usage** : catalogues, contenus, dossiers échangés en JSON, réponses à des questionnaires.

2.3 COLONNES

Les données sont rangées par **clé de ligne**, puis par **familles de colonnes**. Chaque ligne peut avoir ses propres colonnes, et seules les cellules renseignées sont stockées. Les lignes sont triées par clé : lire une plage de clés voisines est très rapide. Chaque cellule peut garder plusieurs versions horodatées.

- **Produits** : Apache HBase (sur HDFS), Apache Cassandra, Google Bigtable.
- **Points forts** : écritures massives, milliards de lignes, lecture par clé ou par plage de clés.
- **Limites** : la clé de ligne doit être pensée pour la requête ; une recherche sur une autre colonne parcourt toute la table.
- **Cas d'usage** : historiques et séries temporelles, journaux d'événements, parcours d'un client ou d'un patient.

Il ne faut pas confondre cette famille avec les **formats colonnes** ORC et Parquet utilisés par Hive : ces derniers rangent les valeurs d'une même colonne ensemble pour accélérer les agrégations, alors que HBase sert surtout des lectures par clé.

2.4 GRAPHES

Les données sont des nœuds (un patient, un médecin, un établissement) reliés par des relations orientées (*consulte*, *exerce dans*). Nœuds et relations portent des propriétés. Parcourir une relation ne demande pas de jointure : chaque nœud pointe directement vers ses voisins.

- **Produits** : Neo4j (langage Cypher), JanusGraph, Amazon Neptune.
- **Points forts** : requêtes sur les chemins et les voisinages, même à plusieurs niveaux.
- **Limites** : mal adaptée aux agrégations sur tout le jeu de données, répartition sur plusieurs serveurs difficile.
- **Cas d'usage** : réseaux sociaux, recommandation, détection de fraude, réseau de soins (qui adresse ses patients à qui).

2.5 TABLEAU COMPARATIF

	Clé-valeur	Documents	Colonnes	Graphes
Unité stockée	Paire clé, valeur	Document JSON / BSON	Ligne : clé, familles de colonnes	Nœuds et relations
Schéma	Aucun	Souple, par document	Familles fixées, colonnes libres	Souple
Accès type	Par clé	Par champs, index secondaires	Par clé ou plage de clés	Parcours de relations
Passage à l'échelle	Très bon	Bon (sharding)	Très bon	Plus difficile
Produits	Redis, DynamoDB	MongoDB, CouchDB	HBase, Cassandra	Neo4j, JanusGraph
Exemple CHU	Cache des indicateurs du tableau de bord	Réponses brutes aux questionnaires e-Satis	Parcours de soins d'un patient	Réseau patients, professionnels, établissements

3 SGBDR ET NOSQL

3.1 ACID ET BASE

Un SGBD relationnel garantit les propriétés ACID : c'est indispensable quand une écriture ne doit jamais être perdue ni appliquée à moitié (un virement, une prescription). Ces garanties ont un coût : verrous, journal de transactions, coordination entre serveurs. Elles rendent difficile la répartition sur un grand nombre de machines.

Beaucoup de bases NoSQL choisissent le modèle BASE : elles acceptent qu'une lecture renvoie pendant un court instant une ancienne valeur, en échange d'une disponibilité et d'un débit plus élevés. Ce choix n'est pas absolu : MongoDB et HBase offrent des écritures atomiques sur un document ou une ligne, et Cassandra laisse régler le niveau de cohérence requête par requête.

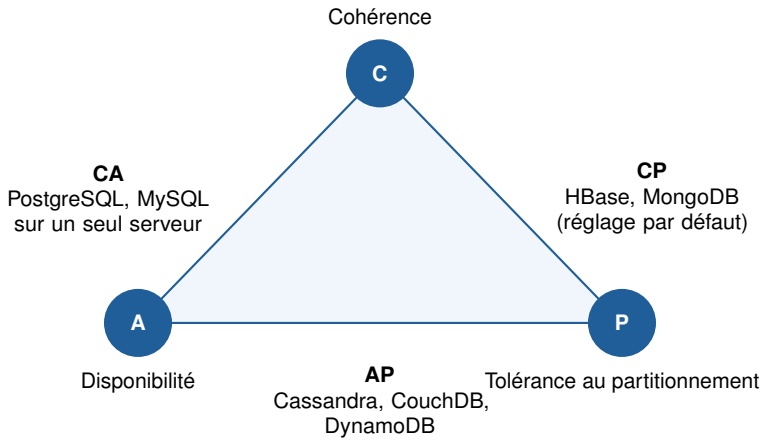
	ACID	BASE
Priorité	Exactitude de chaque transaction	Disponibilité et débit
Cohérence	Immédiate, forte	À terme (*eventual*)
Transactions	Sur plusieurs lignes et tables	Souvent limitées à une ligne ou un document
Répartition	Difficile au-delà de quelques serveurs	Conçue pour des dizaines ou centaines de nœuds
Exemples	PostgreSQL, Oracle, SQL Server	Cassandra, DynamoDB, CouchDB

3.2 LE THÉORÈME CAP

Sur un système réparti, la tolérance au partitionnement n'est pas une option : un câble ou un commutateur finira par tomber. Pendant la coupure, chaque partie du cluster doit choisir :

- **CP** : refuser de répondre (ou d'écrire) tant que la cohérence n'est pas garantie ;
- **AP** : répondre quand même, quitte à renvoyer une valeur qui n'est plus à jour.

Le cas **CA** ne concerne en pratique qu'un système sur un seul serveur, où aucune coupure interne n'est possible.



Système	Famille	Choix	Comportement pendant une coupure
PostgreSQL (un serveur)	Relationnel	CA	Pas de coupure interne possible ; si le serveur tombe, plus de service
HBase	Colonnes	CP	Chaque plage de clés n'a qu'un serveur responsable : elle est indisponible le temps de sa réaffectation
MongoDB	Documents	CP	Seul le primaire accepte les écritures ; sans majorité, pas de nouvelle écriture
Cassandra	Colonnes	AP	Tous les nœuds acceptent les écritures, réconciliées ensuite ; cohérence réglable
CouchDB	Documents	AP	Réplication multi-maîtres, conflits résolus après coup

Ce classement décrit le réglage par défaut. Plusieurs produits permettent de déplacer le curseur (niveau de cohérence de Cassandra, lectures sur les secondaires de MongoDB).

3.3 COMPARAISON

Critère	SGBDR	NoSQL
Modèle	Tables liées par des clés	Clé-valeur, documents, colonnes ou graphes
Schéma	Fixe, défini avant l'écriture	Souple, souvent défini à la lecture
Langage	SQL normalisé	API propre à chaque produit (CQL, Cypher, requêtes JSON)
Propriétés	ACID	BASE le plus souvent
Passage à l'échelle	Vertical (un serveur plus puissant)	Horizontal (plus de serveurs)
Jointures	Natives et optimisées	Absentes ou limitées : on duplique les données
Conception	À partir des données (normalisation)	À partir des requêtes à servir
Points forts	Intégrité, requêtes libres, maturité	Volume, débit, souplesse, disponibilité

Hive n'entre dans aucune des deux colonnes : il offre du SQL et un schéma fixe, comme un SGBDR, mais sur des fichiers répartis dans HDFS et pour des lectures en masse. Il ne remplace ni la base opérationnelle ni une base NoSQL d'accès rapide.

3.4 CHOISIR LE STOCKAGE ADAPTÉ À UN BESOIN MÉTIER

Le choix part de la façon dont les données seront **lues** :

- des **agrégations** sur des millions de lignes, selon des axes variés → entrepôt en colonnes (Hive avec ORC) ;
- des **transactions** fréquentes et sûres sur des données liées → SGBDR ;
- la lecture de **tout ce qui concerne un élément** (un patient), très vite → base colonnes ou documents, avec une clé bien choisie ;
- des **relations** à parcourir sur plusieurs niveaux → base graphe ;
- une **valeur** à relire des milliers de fois → cache clé-valeur.

Appliqué au CHU :

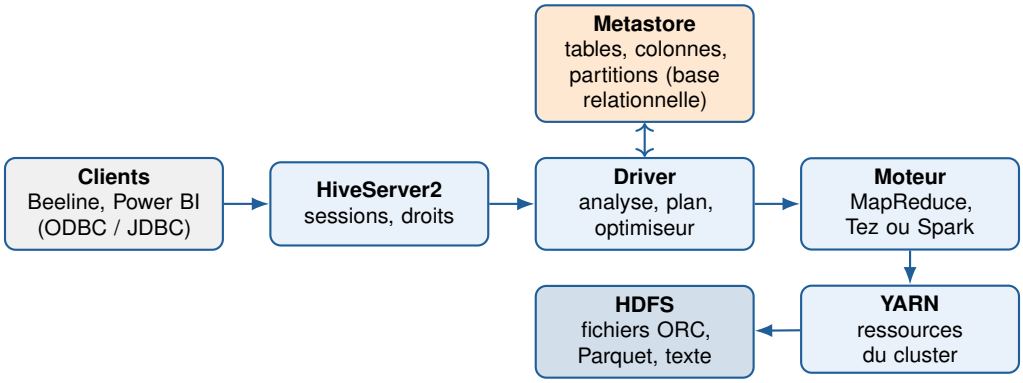
Besoin	Stockage	Justification
Les 8 analyses demandées (taux de consultation, d'hospitalisation, décès, satisfaction)	Hive, tables ORC partitionnées	Agrégations sur des millions de lignes, par période et par axe ; SQL connu des analystes, branché à Power BI
Dossier patient au quotidien (saisie des soins)	SGBDR (PostgreSQL, base de soins existante)	Écritures fréquentes, intégrité indispensable, transactions ACID
Parcours de soins d'un patient	HBase (choix du CER 2)	Lecture par clé en quelques millisecondes, déjà présent dans Cloudera, données dans HDFS
Réponses brutes aux questionnaires de satisfaction	Base documents (MongoDB)	Questionnaires qui changent d'une année à l'autre : schéma souple
Réseau d'adressage entre professionnels	Base graphe (Neo4j)	Chemins entre médecins, établissements et patients ; besoin exploratoire, non demandé à ce stade
Indicateurs affichés en page d'accueil	Cache clé-valeur (Redis)	Valeur recalculée tous les trois mois et relue très souvent

Pour le livrable, seul l'entrepôt Hive est nécessaire : les 8 besoins sont tous des agrégations. Les lignes suivantes du tableau montrent que le CHU aura à terme plusieurs stockages, chacun pour un type d'accès (on parle de **persistance polyglotte**).

4 HIVE ET MAPREDUCE

4.1 ARCHITECTURE DE HIVE

Hive ne stocke pas les données lui-même : il décrit des fichiers déjà présents dans HDFS et transforme les requêtes HiveQL en traitements distribués.



Une requête suit le chemin suivant :

1. le client envoie la requête à **HiveServer2** ;
2. le **driver** l'analyse et consulte le **metastore** pour connaître les colonnes, l'emplacement des fichiers et la liste des partitions ;
3. l'**optimiseur** produit un plan : partitions à lire, ordre et type des jointures, filtres poussés au plus près des fichiers ;
4. le plan devient un ensemble de tâches (MapReduce, ou un graphe de tâches Tez) lancées par **YARN** sur les nœuds qui portent les blocs HDFS ;
5. le résultat remonte au client.

Le principe est le **schéma à la lecture** : Hive ne vérifie pas les fichiers au chargement, il les interprète au moment de la requête. La commande `EXPLAIN` affiche le plan, ce qui permet de vérifier qu'une partition est bien élaguée ou qu'une jointure est faite en mémoire.

Le moteur d'exécution dépend de la distribution : MapReduce était le moteur d'origine ; Tez, plus rapide car il enchaîne les étapes sans écrire de fichiers intermédiaires sur HDFS, est le moteur par défaut de Hive 3 (Cloudera CDP) ; les anciennes distributions Cloudera (CDH) proposent MapReduce ou Spark.

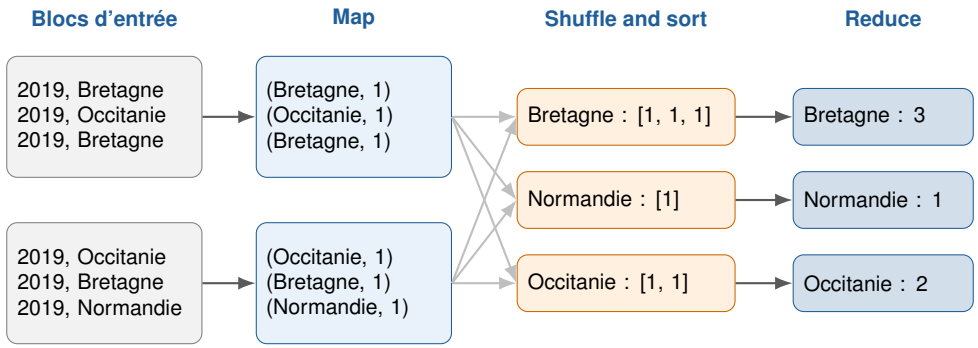
4.2 FONCTIONNEMENT DE MAPREDUCE

Un traitement MapReduce se déroule en quatre temps :

1. **découpage** : le fichier d'entrée est découpé en blocs (*splits*), en général un par bloc HDFS de 128 Mo ;
2. **map** : une tâche par bloc lit les lignes et émet des paires (clé, valeur). Un **combiner** facultatif fait une première agrégation locale pour réduire le volume envoyé sur le réseau ;
3. **shuffle and sort** : les paires sont envoyées vers les reducers selon un hachage de la clé, puis triées et regroupées par clé ;
4. **reduce** : chaque reducer reçoit une clé et la liste de ses valeurs, les agrège et écrit le résultat dans HDFS.

4.3 EXEMPLE : NOMBRE DE DÉCÈS PAR RÉGION EN 2019

Le besoin 7 se prête bien à un exemple. On suppose que la région de décès est déjà jointe à chaque ligne (la petite dimension des régions est copiée sur chaque nœud et la jointure se fait dans le map). Le schéma ci-dessous suit six lignes fictives, réparties en deux blocs.



Source : exemple fictif de six lignes, réalisé pour ce CER

En pseudo-code, le traitement tient en deux fonctions :

```
map(cle, ligne_deces):
    si annee(ligne_deces.id_temps) == 2019:
        region = regions[ligne_deces.id_region_deces]   -- dimension en memoire
        emettre(region, 1)

reduce(region, valeurs):
    emettre(region, somme(valeurs))
```

La somme étant associative, la fonction reduce peut aussi servir de combiner : chaque map n'envoie alors qu'une paire par région au lieu d'une paire par décès. Sur les 25 millions de lignes du fichier INSEE, c'est ce qui limite le trafic réseau.

Dans Hive, on n'écrit pas ce code : la requête du besoin 7 (partie 6.4) est compilée en un plan équivalent. Le `WHERE` devient un filtre dans le map, le `JOIN` avec la petite dimension une jointure en mémoire (*map join*), et le `GROUP BY` un shuffle suivi d'un reduce. Ce traitement n'a pas été exécuté sur le cluster : l'exemple a été déroulé à la main.

5 OPTIMISER LE MODÈLE PHYSIQUE DANS HIVE

Une requête Hive est lente surtout parce qu'elle **lit trop de données**, ou parce qu'elle les **déplace** sur le réseau pendant les jointures. Les leviers d'optimisation visent l'un ou l'autre.

5.1 PARTITIONNEMENT

Une table partitionnée est rangée en un sous-répertoire par valeur de la colonne de partition. Cette colonne n'est pas stockée dans les fichiers : elle est portée par le nom du répertoire.

```
/warehouse/entrepot.db/fait_deces/
    annee=1970/000000_0
    annee=1971/000000_0
    ...
    annee=2019/000000_0      <- seul répertoire lu si WHERE annee = 2019
    annee=2020/000000_0
```

Deux façons de remplir les partitions :

- **statique** : la valeur est écrite dans la requête (`PARTITION (annee=2019)`). Adapté au chargement d'une seule période connue ;
- **dynamique** : Hive crée les partitions d'après la dernière colonne du `SELECT` (`PARTITION (annee)`). Adapté au chargement initial de tout l'historique. Il faut l'autoriser : `hive.exec.dynamic.partition=true` et `hive.exec.dynamic.partition.mode=nonstrict`.

Choisir la colonne de partition :

- une colonne présente dans les filtres de la plupart des requêtes (la période, ici l'année) ;
- une **cardinalité faible** : quelques dizaines à quelques centaines de valeurs, pas l'identifiant patient ni la date au jour ;
- des partitions assez grosses : idéalement plusieurs centaines de mégaoctets chacune.

Le piège est le **sur-partitionnement** : trop de petites partitions produisent des milliers de petits fichiers. Le NameNode garde chaque fichier en mémoire, le metastore doit lister toutes les partitions, et chaque fichier lance une tâche. Une requête peut alors devenir plus lente qu'avec une table non partitionnée. Hive limite d'ailleurs le nombre de partitions dynamiques créées par requête (`hive.exec.max.dynamic.partitions`, 1 000 par défaut).

L'élagage ne fonctionne que si le filtre porte **directement** sur la colonne de partition. Un filtre sur `dim_temps.annee` après jointure n'élague rien si la table de faits est partitionnée sur sa propre colonne `annee` : il faut filtrer `f.annee`.

5.2 BUCKETING

Le bucketing découpe une table, ou chaque partition, en un nombre fixe de fichiers, selon le hachage d'une colonne :

```
CREATE TABLE entrepot.dim_etablissement ( ... )
CLUSTERED BY (id_etablissement) SORTED BY (id_etablissement) INTO 4 BUCKETS
STORED AS ORC;
```

Il sert à trois choses :

- **bucket map join** : si deux tables sont découpées sur leur clé de jointure, avec des nombres de buckets multiples l'un de l'autre, chaque tâche ne charge en mémoire que le bucket correspondant de l'autre table (`hive.optimize.bucketmapjoin=true`) ;
- **sort-merge bucket join** (SMB) : si les deux tables ont le **même** nombre de buckets et sont aussi triées sur la clé (`SORTED BY`), la jointure se fait par simple fusion de fichiers triés, sans shuffle ni table en mémoire (`hive.optimize.bucketmapjoin.sortedmerge=true`, `hive.auto.convert.sortmerge.join=true`) ;
- **échantillonnage** : `TABLESAMPLE(BUCKET 1 OUT OF 4)` lit un quart de la table pour tester une requête.

Contrairement à la partition, le bucket convient aux colonnes de **forte cardinalité**, comme une clé de jointure. Le nombre de buckets se fixe à la création et ne change plus sans recharger la table : on vise des fichiers proches d'un bloc HDFS, et une puissance de deux. Sur les versions de Hive antérieures à la 2.0, il faut activer `hive.enforce.bucketing=true` avant chaque chargement, sinon les lignes ne sont pas réparties.

5.3 FORMATS COLONNES ET COMPRESSION

Le format texte (CSV) oblige à lire et découper chaque ligne entière. Les formats colonnes **ORC** et **Parquet** rangent ensemble les valeurs d'une même colonne :

- une requête ne lit que les colonnes dont elle a besoin ;
- des valeurs semblables, côte à côte, se compressent bien (ZLIB, Snappy) ;
- ORC stocke le minimum et le maximum de chaque colonne par bloc de lignes : un filtre peut sauter les blocs inutiles (*predicate pushdown*), et des filtres de Bloom peuvent être ajoutés sur une colonne.

ORC est le format natif de Hive et le seul qui supporte ses tables transactionnelles ; Parquet est préféré quand les fichiers sont aussi lus par Spark ou Impala.

5.4 STATISTIQUES, VECTORISATION ET OPTIMISEUR

- **Statistiques** : `ANALYZE TABLE ... COMPUTE STATISTICS` enregistre dans le metastore le nombre de lignes, et `FOR COLUMNS` le nombre de valeurs distinctes, les nulls, le minimum et le maximum. Certaines requêtes (`COUNT(*)`) sont alors servies sans lire les fichiers (`hive.compute.query.using.stats`).
- **Optimiseur fondé sur les coûts** (CBO, avec Apache Calcite) : grâce aux statistiques, il choisit l'ordre des jointures et décide quelles tables sont assez petites pour une jointure en mémoire (`hive.cbo.enable`, `hive.auto.convert.join`).
- **Vectorisation** : les opérateurs traitent les lignes par lots de 1 024 au lieu d'une à la fois, ce qui réduit le coût processeur des filtres et des agrégations. Elle fonctionne avec ORC (`hive.vectorized.execution.enabled`).
- **Moteur Tez** : il enchaîne les étapes d'une requête en un seul graphe, sans écrire les résultats intermédiaires sur HDFS comme le fait MapReduce entre deux jobs (`hive.execution.engine=tez`).

5.5 VUES MATÉRIALISÉES

Une vue matérialisée stocke le résultat d'une agrégation. Depuis Hive 3, l'optimiseur peut **réécrire automatiquement** une requête pour lire la vue au lieu des tables de faits, sans que l'utilisateur change sa requête :

```
CREATE MATERIALIZED VIEW entrepot.mv_exemple
STORED AS ORC
AS SELECT ... GROUP BY ...;

ALTER MATERIALIZED VIEW entrepot.mv_exemple REBUILD;  -- après chaque chargement
```

Points d'attention :

- une vue périmée n'est pas utilisée pour la réécriture ; il faut la reconstruire (`REBUILD`) après chaque chargement trimestriel ;
- Hive ne sait dire si la vue est à jour, et ne peut la reconstruire de façon incrémentale, que si les tables sources sont **transactionnelles** ;
- la réécriture se contrôle par `hive.materializedview.rewriting` ; une vue peut aussi être créée avec `DISABLE REWRITE` et interrogée directement.

Sur une version de Hive antérieure à la 3, le même effet s'obtient avec une **table d'agrégats** ordinaire, rechargée par `INSERT OVERWRITE` à chaque mise à jour, et interrogée explicitement.

5.6 SYNTHÈSE DES LEVIERS

Levier	Effet attendu	Condition	Hive
Partitionnement	Ne lit que les périodes filtrées	Filtre direct sur la colonne, partitions pas trop petites	0.x
Bucketing	Jointures sans shuffle (bucket map join, SMB)	Même clé et nombre de buckets compatible des deux côtés	0.x
ORC + compression	Moins d'octets lus, colonnes utiles seulement	Recharger les tables au format ORC	0.11
Statistiques + CBO	Meilleur plan, jointures en mémoire	`ANALYZE TABLE` après chaque chargement	0.14
Vectorisation	Moins de temps processeur	Format ORC	0.13
Tez	Moins d'écritures intermédiaires	Distribution qui fournit Tez	0.13
Vue matérialisée	Agrégat précalculé, réécriture automatique	Hive 3, reconstruction après chargement	3.0

Source : Apache Hive Language Manual et pages de conception du wiki Apache Hive

La dernière colonne donne la version de Hive qui a introduit la fonctionnalité. La première étape sur le cluster sera donc de relever sa version (`hive -version`, ou `SELECT version();` dans Beeline à partir de Hive 2.1), pour savoir quels leviers sont disponibles.

6 APPLICATION AU CHU : MODÈLE PHYSIQUE DANS HIVE

Le modèle dimensionnel est la constellation du projet : quatre tables de faits (consultations, hospitalisations, satisfaction, décès) qui partagent des dimensions communes (temps, établissement, région, diagnostic, patient, professionnel). Il ne change pas. Seule sa traduction physique dans Hive est décidée ici. Les jobs Talend J00 à J10 alimentent l'entrepôt : PostgreSQL sert de cible de développement, Hive sur HDFS (Cloudera) est la cible finale. Ce qui suit n'a pas été exécuté sur le cluster.

6.1 VOLUMES ET CHOIX PAR TABLE

Table	Volume	Période	Choix physique
fait_deces	25 088 208 lignes	1970–2020	ORC, partition par année de décès (51 partitions)
fait_consultation	1 027 157 lignes	2015–2023	ORC, partition par année (9), 4 buckets sur `id_etablissement`
fait_hospitalisation	2 479 lignes	2015–2021	ORC, ni partition ni bucket
fait_satisfaction	environ 2 000 lignes	2020	ORC, partition par année
dim_etablissement	environ 416 000 lignes	–	ORC, 4 buckets triés sur `id_etablissement`
dim_professionnel	1 048 575 lignes	–	ORC
dim_temps, dim_patient, dim_diagnostic, dim_region_insee	de 15 490 à 100 000 lignes	–	ORC, jointures en mémoire

Les choix se justifient ainsi :

- **fait_deces** porte l'essentiel du volume. Le besoin 7 filtre une seule année : avec une partition par année, la requête lit 1 partition sur 51, soit environ 490 000 lignes en moyenne au lieu de 25 millions. Une partition par mois donnerait plus de 600 partitions bien plus petites, sans gain pour les besoins actuels.
- **fait_consultation** est filtré par période (besoins 1, 2 et 6) : la partition par année découpe environ 114 000 lignes par année. Le bucket sur `id_etablissement`, avec le même découpage sur la dimension établissement, permet une jointure par buckets pour le besoin 1. L'établissement est choisi parce que c'est la dimension partagée par trois faits.
- **fait_hospitalisation** est trop petite pour être découpée : 7 partitions d'environ 350 lignes ne produiraient que de petits fichiers. Un seul fichier ORC est lu en entier en une fraction de seconde ; le choix sera revu si le volume grandit.
- **fait_satisfaction** n'a qu'une année aujourd'hui. La partition par année sert surtout le chargement : chaque nouvelle campagne e-Satis s'ajoute dans sa partition sans réécrire les autres.
- Les **petites dimensions** tiennent en mémoire : Hive les diffuse à chaque tâche (*map join*). Les découper n'apporterait rien.

Le bucket a peu d'effet sur un fait d'un million de lignes : il est surtout prévu pour la suite, quand les consultations s'ajouteront tous les trois mois. Il ne sera conservé que si la mesure montre un gain.

6.2 CRÉATION DES TABLES

Faits Décès et Consultation

```
CREATE DATABASE IF NOT EXISTS entrepot;

CREATE TABLE entrepot.fait_deces (
    id_deces             BIGINT,
    id_temps             INT,          -- date de décès AAAAMMJJ
    id_region_deces      INT,
    id_region_naissance  INT,
    age_deces            INT,
    annee_naissance      INT,
    numero_acte_deces    STRING,
    flag_date_partielle  BOOLEAN,
    date_chargement      TIMESTAMP
)
PARTITIONED BY (annee INT)               -- année de décès
STORED AS ORC
TBLPROPERTIES ('orc.compress' = 'SNAPPY');

CREATE TABLE entrepot.fait_consultation (
    num_consultation        INT,
    id_patient              INT,
    id_temps                INT,
    id_professionnel        INT,
    id_diagnostic           INT,
    id_etablissement        INT,
    nombre_de_consultation  INT
)
PARTITIONED BY (annee INT)
CLUSTERED BY (id_etablissement) SORTED BY (id_etablissement) INTO 4 BUCKETS
STORED AS ORC
TBLPROPERTIES ('orc.compress' = 'SNAPPY');
```

Faits Hospitalisation et Satisfaction

```
CREATE TABLE entrepot.fait_hospitalisation (
    id_hospitalisation    INT,
    id_patient            INT,
    id_etablissement      INT,
    id_diagnostic         INT,
    id_date_entree        INT,            -- AAAAMMJJ
    num_hospitalisation   STRING,
    jour_hospitalisation  INT
)
STORED AS ORC;                           -- 2 479 lignes : pas de découpage

CREATE TABLE entrepot.fait_satisfaction (
    id_satisfaction            INT,
    id_etablissement           INT,
    id_region                  INT,
    participation_obligatoire  BOOLEAN,
    depot_valide               BOOLEAN,
    nb_reponses                INT,
    score                      DECIMAL(5,2)
)
PARTITIONED BY (annee INT)               -- une campagne e-Satis par année
STORED AS ORC;
```

Dans Hive, la colonne de partition ne figure pas dans la liste des colonnes : `annee` de la satisfaction passe donc dans la clause `PARTITIONED BY`. Hive ne vérifie ni clé primaire ni clé étrangère : l'intégrité est contrôlée par les jobs Talend avant le chargement. La dimension établissement est créée comme dans la partie précédente, avec `CLUSTERED BY (id_etablissement) SORTED BY (id_etablissement) INTO 4 BUCKETS`.

6.3 CHARGEMENT DEPUIS LES TABLES DE TRANSIT

Les jobs Talend déposent les données préparées dans des tables de transit (base `transit`). Le passage vers l'entrepôt se fait en HiveQL, avec un partitionnement dynamique :

```
SET hive.exec.dynamic.partition = true;
SET hive.exec.dynamic.partition.mode = nonstrict;

INSERT OVERWRITE TABLE entrepot.fait_deces PARTITION (annee)
SELECT id_deces, id_temps, id_region_deces, id_region_naissance,
       age_deces, annee_naissance, numero_acte_deces,
       flag_date_partielle, date_chargement,
       id_temps DIV 10000 AS annee        -- dernière colonne = partition
FROM transit.fait_deces;

INSERT OVERWRITE TABLE entrepot.fait_consultation PARTITION (annee)
SELECT num_consultation, id_patient, id_temps, id_professionnel,
       id_diagnostic, id_etablissement, nombre_de_consultation,
       id_temps DIV 10000 AS annee
FROM transit.fait_consultation;

ANALYZE TABLE entrepot.fait_deces PARTITION (annee) COMPUTE STATISTICS;
ANALYZE TABLE entrepot.fait_deces PARTITION (annee) COMPUTE STATISTICS FOR COLUMNS;
```

Trois remarques :

- l'année est déduite de `id_temps` (`AAAAMMJJ`) par division entière ; les dates partielles du fichier INSEE gardent leur année, signalée par `flag_date_partielle` ;
- avec des partitions dynamiques, `INSERT OVERWRITE` ne remplace **que** les partitions présentes dans le résultat. À chaque mise à jour trimestrielle, il suffit donc de charger les lignes des années concernées pour réécrire ces seules années ;
- les statistiques sont recalculées après chaque chargement, sinon l'optimiseur travaille sur des chiffres faux.

6.4 REQUÊTES DES BESOINS

Les réglages suivants sont fixés au début de chaque session (ou dans la configuration du cluster) :

```
SET hive.execution.engine = tez;                -- si la distribution le fournit
SET hive.vectorized.execution.enabled = true;
SET hive.cbo.enable = true;
SET hive.compute.query.using.stats = true;
SET hive.stats.fetch.column.stats = true;
SET hive.auto.convert.join = true;              -- petites dimensions en mémoire
SET hive.optimize.bucketmapjoin = true;
SET hive.optimize.bucketmapjoin.sortedmerge = true;
SET hive.auto.convert.sortmerge.join = true;
```

Besoin 1 : taux de consultation par établissement et par période. On retient ici comme taux la part des consultations d'une période réalisées dans chaque établissement.

```
WITH c AS (
    SELECT f.annee, t.trimestre, e.id_etablissement, e.raison_sociale_site,
           SUM(f.nombre_de_consultation) AS nb_consultations
    FROM entrepot.fait_consultation f
    JOIN entrepot.dim_temps t         ON t.id_temps = f.id_temps
    JOIN entrepot.dim_etablissement e ON e.id_etablissement = f.id_etablissement
    WHERE f.annee BETWEEN 2019 AND 2023          -- élagage des partitions
    GROUP BY f.annee, t.trimestre, e.id_etablissement, e.raison_sociale_site
)
SELECT annee, trimestre, raison_sociale_site, nb_consultations,
       ROUND(100 * nb_consultations
             / SUM(nb_consultations) OVER (PARTITION BY annee, trimestre), 2)
           AS taux_pct
FROM c
ORDER BY annee, trimestre, taux_pct DESC;
```

Besoin 4 : taux d'hospitalisation par diagnostic et par période, c'est-à-dire la part des séjours de l'année dus à chaque diagnostic.

```
WITH h AS (
    SELECT t.annee, d.code_diagnostic, d.diagnostic, COUNT(*) AS nb_sejours
    FROM entrepot.fait_hospitalisation f
    JOIN entrepot.dim_temps t      ON t.id_temps = f.id_date_entree
    JOIN entrepot.dim_diagnostic d ON d.id_diagnostic = f.id_diagnostic
    GROUP BY t.annee, d.code_diagnostic, d.diagnostic
)
SELECT annee, code_diagnostic, diagnostic, nb_sejours,
       ROUND(100 * nb_sejours / SUM(nb_sejours) OVER (PARTITION BY annee), 2)
           AS taux_pct
FROM h
ORDER BY annee, taux_pct DESC;
```

Besoin 7 : nombre de décès par région en 2019.

```
SELECT r.code_region, r.nom_region, COUNT(*) AS nb_deces
FROM entrepot.fait_deces f
JOIN entrepot.dim_region_insee r ON r.id_region = f.id_region_deces
WHERE f.annee = 2019                    -- 1 partition lue sur 51
GROUP BY r.code_region, r.nom_region
ORDER BY nb_deces DESC;
```

Besoin 8 : satisfaction globale par région en 2020. Le score moyen est pondéré par le nombre de réponses, pour qu'un établissement avec peu de répondants ne pèse pas autant qu'un grand ; seuls les dépôts valides sont retenus.

```
SELECT r.code_region, r.nom_region,
       ROUND(SUM(s.score * s.nb_reponses) / SUM(s.nb_reponses), 2)
           AS score_moyen_pondere,
       SUM(s.nb_reponses) AS nb_reponses,
       COUNT(*)           AS nb_evaluations
FROM entrepot.fait_satisfaction s
JOIN entrepot.dim_region_insee r ON r.id_region = s.id_region
WHERE s.annee = 2020 AND s.depot_valide = true
GROUP BY r.code_region, r.nom_region
ORDER BY score_moyen_pondere DESC;
```

Dans les quatre requêtes, le filtre de période porte directement sur la colonne de partition du fait (`f.annee`, `s.annee`) quand elle existe, pour que Hive élague les partitions. Pour le besoin 4, la table est assez petite pour être lue en entier.

6.5 VUE MATÉRIALISÉE

Les décès par région et par année sont une agrégation stable : le fichier INSEE n'est mis à jour que tous les trois mois. Une vue matérialisée la précalcule une fois :

```
CREATE MATERIALIZED VIEW entrepot.mv_deces_region_annee
STORED AS ORC
AS
SELECT f.annee, r.code_region, r.nom_region, COUNT(*) AS nb_deces
FROM entrepot.fait_deces f
JOIN entrepot.dim_region_insee r ON r.id_region = f.id_region_deces
GROUP BY f.annee, r.code_region, r.nom_region;

-- après chaque chargement trimestriel
ALTER MATERIALIZED VIEW entrepot.mv_deces_region_annee REBUILD;
```

La vue contient une ligne par année et par région, de l'ordre du millier de lignes au lieu de 25 millions. La requête du besoin 7 reste inchangée : si la vue est à jour, l'optimiseur la réécrit pour lire la vue (`EXPLAIN` doit alors citer `mv_deces_region_annee`). Sur Hive 3 (Cloudera CDP), une table gérée au format ORC est transactionnelle par défaut, ce qui permet à Hive de savoir si la vue est à jour. Sur une version plus ancienne, la vue est remplacée par une table d'agrégats rechargée par `INSERT OVERWRITE`.

6.6 PROTOCOLE DE MESURE

La contrainte est un temps de réponse inférieur à 2 minutes par analyse. Aucune mesure n'a pu être faite : le cluster n'était pas disponible pour ce travail. Le protocole prévu est le suivant :

1. **relever l'environnement** : version de Hive, moteur (MapReduce ou Tez), nombre de nœuds et mémoire allouée ;
2. **situation de départ** : requêtes sur les tables de transit (texte, sans partition, sans bucket, sans statistiques) ;
3. **situation optimisée** : tables ORC partitionnées et découpées, statistiques calculées, réglages ci-dessus, vue matérialisée ;
4. **vérifier le plan** avec `EXPLAIN` avant de mesurer : partitions lues, type de jointure (map join, SMB), réécriture par la vue ;
5. **exécuter** chaque requête une fois à froid (temps noté à part), puis trois fois ; on retient la médiane des trois. Le temps est celui affiché par Beeline, complété par les compteurs du job (octets lus sur HDFS) ;
6. **changer un seul levier à la fois** (format, puis partition, puis bucket, puis vue) pour savoir d'où vient le gain ;
7. **conclure** : chaque besoin dont la médiane dépasse 120 secondes reçoit une optimisation supplémentaire (table d'agrégats, autre colonne de partition).

Besoin	Effet attendu de l'optimisation	Avant (s)	Après (s)	Partitions lues	< 2 min
1	5 partitions sur 9, jointure par buckets	à mesurer	à mesurer	à relever	–
4	Petite table, dimensions en mémoire	à mesurer	à mesurer	–	–
7	1 partition sur 51, ou lecture de la vue	à mesurer	à mesurer	à relever	–
8	1 partition, dimension en mémoire	à mesurer	à mesurer	à relever	–

Les effets de la deuxième colonne sont **attendus**, déduits du fonctionnement de Hive ; seule l'exécution sur le cluster dira si le seuil de 2 minutes est tenu.

7 INTÉGRER DES DONNÉES DANS UNE BASE NOSQL AVEC UN ETL

Le CER 2 a retenu **HBase** pour le parcours de soins d'un patient : il est fourni par Cloudera et stocke ses données dans HDFS. Le job Talend qui l'alimente reprend les mêmes sources que l'entrepôt. Les composants NoSQL (`tHBaseConnection`, `tHBaseOutput`, `tMongoDBConnection`, `tMongoDBOutput`) sont fournis par l'édition Big Data de Talend Open Studio.

7.1 CONCEVOIR LA CLÉ DE LIGNE

Dans HBase, tout repose sur la clé de ligne : c'est le seul accès rapide, et les lignes sont triées par clé. Pour lire le parcours d'un patient, du plus récent au plus ancien :

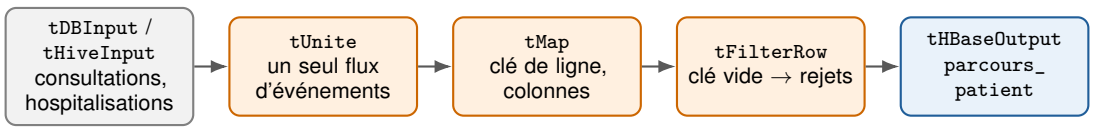
```
<id_patient_hash>#<99999999 - AAAAMMJJ>#<type><numero>
exemple : 3f9a...e1#79849698#C1027157
```

- le **préfixe patient** regroupe tous les événements d'un patient ; comme c'est un hachage, les patients se répartissent uniformément entre les serveurs, sans point chaud ;
- la **date inversée** range les événements du plus récent au plus ancien ;
- le **type et le numéro** (C pour consultation, H pour hospitalisation) rendent la clé unique quand deux événements tombent le même jour.

La table est créée à l'avance, découpée en régions sur les premiers caractères du hachage pour que le chargement initial soit réparti dès le départ :

```
create 'parcours_patient', 'evt', 'lieu', {SPLITS => ['4', '8', 'c']}
```

7.2 LE JOB TALEND



- `tHBaseConnection` indique le quorum ZooKeeper du cluster et son port ;
- dans `tHBaseOutput`, chaque colonne du schéma est rattachée à une famille : date, type et diagnostic dans `evt`, établissement et professionnel dans `lieu` ;
- une écriture HBase avec une clé existante remplace la cellule (une nouvelle version) : relancer le job ne crée pas de doublon, ce qui convient au rechargement trimestriel ;
- le contrôle compare le nombre d'événements lus au nombre de lignes écrites (`count 'parcours_patient'` dans le shell HBase), et la lecture d'un parcours se teste avec un `scan` sur le préfixe d'un patient.

Variante MongoDB : avec `tMongoDBOutput`, on écrirait un document par événement, avec un index sur `id_patient_hash`. Un document unique par patient, avec la liste de ses événements, grossirait sans limite à chaque chargement, alors qu'un document MongoDB est plafonné à 16 Mo.

Ce job est conçu mais n'a pas été exécuté sur le cluster : il reste à le tester sur la machine Cloudera.

CONCLUSION

En réponse à la problématique, implémenter l'entrepôt du CHU dans Hive revient à traduire la constellation en un modèle physique pensé pour les requêtes. Chaque table de faits est stockée en **ORC** compressé, pour ne lire que les colonnes utiles. Les faits volumineux et filtrés par période sont **partitionnés par année** : la requête des décès de 2019 ne lit plus qu'une partition sur 51. La clé de jointure la plus partagée, l'établissement, sert au **bucketing** du fait des consultations et de sa dimension. Les petites tables ne sont pas découpées, pour éviter les petits fichiers.

Les agrégations stables, comme les décès par région et par année, sont précalculées dans une **vue matérialisée** que Hive 3 utilise de lui-même, à reconstruire après chaque chargement trimestriel. Les statistiques, l'optimiseur fondé sur les coûts, la vectorisation et le moteur Tez complètent ces choix. Le seuil de 2 minutes ne peut pas être affirmé sans mesure : un protocole avant / après, un levier à la fois, est prêt à être exécuté sur le cluster.

Le choix du stockage part de la façon dont les données sont lues. Les huit analyses sont des agrégations : elles relèvent de l'entrepôt Hive. Les bases NoSQL répondent à d'autres accès, chacune selon sa famille : clé-valeur pour un cache, documents pour des données au schéma changeant, colonnes pour lire tout un parcours par clé, graphes pour des relations. Elles échangent souvent une partie des garanties ACID contre la disponibilité, comme l'explique le théorème CAP. Pour le CHU, HBase garde le rôle défini au CER 2 : servir le parcours d'un patient, alimenté par un job Talend.

Enfin, MapReduce éclaire ce que fait Hive : un filtre et une jointure en mémoire dans le map, un regroupement dans le shuffle, une agrégation dans le reduce. Comprendre ce découpage permet de lire un plan d'exécution et de savoir où une requête perd du temps.

CAPITALISATION

AAV	Commentaire	Statut
Distinguer les types de bases NoSQL (organisation, cas d'usage)	Organisation, produits, forces et limites des quatre familles, tableau comparatif et un cas d'usage CHU par famille.	Acquis
Comparer SGBDR et NoSQL (ACID vs BASE, théorème CAP)	ACID et BASE comparés, théorème CAP expliqué avec le classement CP / AP / CA de systèmes courants et ses limites.	Acquis
Utiliser un ETL pour intégrer des données dans une base NoSQL	Clé de ligne HBase conçue (préfixe haché, date inversée), table prédécoupée, job Talend décrit avec ses contrôles. Non exécuté sur le cluster.	Partiel
Sélectionner le type de stockage adapté à un besoin métier	Méthode fondée sur le mode de lecture, appliquée aux besoins du CHU : entrepôt Hive pour les 8 analyses, SGBDR, HBase, documents, graphe et cache pour les autres usages.	Acquis
Utiliser MapReduce pour extraire des informations	Fonctionnement (map, combiner, shuffle and sort, reduce) et exemple du besoin 7 déroulé à la main, lien avec le plan d'une requête Hive. Aucun job exécuté.	Partiel
Optimiser le modèle physique (partitionnement, bucket)	Choix justifiés table par table (partitions, buckets, ORC, vue matérialisée), scripts de création, de chargement et de requêtes, protocole de mesure. Non exécuté sur le cluster : gains non mesurés.	Partiel

RESSOURCES

- **Apache Hive, site officiel** : https://hive.apache.org/
- **Apache Hive, Language Manual** : https://cwiki.apache.org/confluence/display/Hive/LanguageManual
- **Apache Hive, jointures** : https://cwiki.apache.org/confluence/display/Hive/LanguageManual+Joins
- **Apache Hive, vues matérialisées** : https://cwiki.apache.org/confluence/display/Hive/Materialized+views
- **Apache Hive, DDL (partitions)** : https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL
- **Apache Hive, tables en buckets** : https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL+BucketedTables
- **Apache Hive, format ORC** : https://cwiki.apache.org/confluence/display/Hive/LanguageManual+ORC
- **Apache Hive, statistiques** : https://cwiki.apache.org/confluence/display/Hive/StatsDev
- **Apache Hive, exécution vectorisée** : https://cwiki.apache.org/confluence/display/Hive/Vectorized+Query+Execution
- **H. Bansal, S. Chauhan, S. Mehrotra, *Apache Hive Cookbook*, Packt Publishing, 2016**, référencé par le site Apache Hive : https://hive.apache.org/community/resources/books-blogs-talks/
- **Apache Hadoop, tutoriel MapReduce** : https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html
- **E. Brewer, « CAP Twelve Years Later: How the Rules Have Changed », InfoQ** : https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/
- **AWS, différences entre ACID et BASE** : https://aws.amazon.com/compare/the-difference-between-acid-and-base-database/
- **Helpshift Engineering, « Building a Data warehouse with Hive at Helpshift »** : https://medium.com/helpshift-engineering/building-a-data-warehouse-with-hive-at-helpshift-part-1-443046df6484
- **Cloudera, Apache Hive Performance Tuning** : https://docs.cloudera.com/cdp-private-cloud-base/7.1.9/hive-performance-tuning/index.html
- **Apache HBase, guide de référence** : https://hbase.apache.org/book.html
- **Documentation Talend** : https://help.talend.com/